

# PPT-LAT-Roadmap

**Project:** shannon-prime-lattice **Document:** Implementation Roadmap **Audience:** Future Claude sessions and human contributors picking up phases **Status:** Canonical. This document defines the order of work and the gates that separate phases. **Sister documents:** PPT-LAT-Theory.md (the math), PPT-LAT-Systems.md (the architecture).

---

## 0. How to read this document

This is the operational doc. If you are a session opening this file at the start of a working block:

1. Find the lowest-numbered phase whose exit conditions are not all satisfied.
2. Read that phase in full, plus every phase listed in its **Dependencies** line.
3. Read the most recent SESSION-STATE-lat-<phase>.md so you know where the previous session stopped.
4. Re-read the three binding policy sections later in this doc: **The contract system, The offload pattern, Anti-contamination rule.**
5. Begin work. Do not skip ahead — later phases depend on invariants the earlier gates prove.

Each phase is self-contained. You should not need to read source code in D:\F\shannon-prime-repos\shannon-prime\ or D:\F\shannon-prime-repos\shannon-prime-engine\ to do the work.

---

## 1. Phase summary table

| Phase | Goal                                                               | Est. wall-clock (focused dev) | Gate                                                           |
|-------|--------------------------------------------------------------------|-------------------------------|----------------------------------------------------------------|
| 0     | Bootstrap: repos, README, LICENSE, CI scaffolding, contract format | 1 day                         | Three repos exist, push green, CI runs no-op test              |
| 1     | KSTE encoder (pure C reference)                                    | 5-7 days                      | 11/11 unit tests pass on Linux + Windows                       |
| 2     | Dominance order and incomparability sieve                          | 4-5 days                      | 10/10 tests; sieve maintains invariants under random insertion |

| Phase | Goal                                                             | Est. wall-clock (focused dev) | Gate                                                          |
|-------|------------------------------------------------------------------|-------------------------------|---------------------------------------------------------------|
| 3     | ARM (HRR in CRT cyclotomic ring)                                 | 4–5 days                      | 6/6 tests; capacity curve matches Theory section 4            |
| 4     | CRT NTT primitives (dual-prime, Barrett)                         | 5–6 days                      | 5/5 tests; bit-exact vs schoolbook                            |
| 5     | Engine bootstrap: GGUF loader, forward pass, math core wired     | 10–14 days                    | E1: forward pass produces expected logits on canonical input  |
| 6     | Two-node CRT-sharded inference demo                              | 5–7 days                      | E2: bit-identical output to single-machine                    |
| 7     | KSTE-encoded crawl cache (single node)                           | 4–6 days                      | E3: deduplication ratio measured on reference corpus          |
| 8     | DHT + position-as-arithmetic crawl assignment                    | 7–10 days                     | E4: load balancing under skewed URL distribution              |
| 9     | ARM gossip aggregation                                           | 5–7 days                      | E5: aggregated state converges across nodes                   |
| 10    | Verification layer (commitments, dominance proofs, slashing sim) | 7–10 days                     | E6: FP/FN rates under adversarial bindings within target band |

| Phase | Goal                                               | Est. wall-clock (focused dev) | Gate                                                                   |
|-------|----------------------------------------------------|-------------------------------|------------------------------------------------------------------------|
| 11    | Token economy simulator (two-token issuance)       | 5–7 days                      | E7: equilibrium under contributor mixes                                |
| 12    | End-to-end three-node pilot on three real machines | 10–14 days                    | Throughput, latency, dedup ratio, fairness all measured and documented |

Totals: roughly **72-108 focused working days** for the whole stack to walk end-to-end on three machines. The wide band reflects the difficulty of Phase 5 (engine bootstrap) and Phase 10 (verification) more than anything else.

## 2. Phase-by-phase

### Phase 0 — Bootstrap

**Goal.** Stand up the three repositories so subsequent phases have a place to live, a way to be tested, and a fixed naming convention. Administrative, but skipping the unglamorous bits is where technical debt accumulates.

**Deliverables.** - shannon-prime-lattice/ (umbrella repo, PRIVATE on GitHub): - README.md — what the project is, who it is for, how to clone and build the three repos as a set - LICENSE — copy in chosen license verbatim - .gitignore — language-aware, covers C, C++, Python, Rust, CMake build dirs, IDE droppings - papers/ — contains this file plus PPT-LAT-Theory.md, PPT-LAT-Systems.md, and the PPT-ARM/ math reference subfolder - BUILD-ENV.md — exact toolchain versions for Windows (VS2022 + CUDA), Linux (gcc, clang), and any cross-compile targets - NAMING.md — naming conventions (see below) - CONTRACT-FORMAT.md — the template every phase deliverable list follows - .github/workflows/ci.yml — placeholder CI that runs the no-op test - shannon-prime-system/ (math core, PRIVATE on GitHub): - README.md — scope statement, dependencies (none on lattice or engine), license - CMakeLists.txt — top-level, builds nothing yet but sets the project name, C11 standard, warnings-as-errors - src/, include/, tests/ empty dirs with .gitkeep - .github/workflows/ci.yml — runs cmake -B build && cmake --build build && ctest --test-dir build (will pass with no tests for now) - shannon-prime-system-engine/ (engine, PRIVATE on GitHub): - same skeleton as shannon-prime-system - declares dependency on shannon-prime-system via CMake FetchContent or submodule (pick one in Phase 0 and stick with it)

**Naming conventions** (write to NAMING.md): - C symbol prefix is sp\_ for the math core and spe\_ for the engine. - File names are lowercase with underscores: sp\_kste.c, sp\_arm.c, spe\_loader.c. - Public headers expose only sp\_<module>\_\* or spe\_<module>\_\* symbols. Static helpers in .c files are not prefixed. - Tests live in tests/test\_<module>.c and register with CTest one test target per module. - Document everything in Markdown only. No reStructuredText, no Sphinx. Long-form goes in papers/.

**Contract format** (write to CONTRACT-FORMAT.md):

## Phase N — <name>

- Deliverables:
  - <repo>/<path>: <what it does>
  - <repo>/<path>: <what it does>
- Tests (all must pass):
  - T<N>.<k>: <one-line description, what it proves>
- Exit conditions: <bullet list of measurable assertions>

A phase is complete only when every file in the deliverables list exists in the named repo on the main branch and every named test passes in CI.

**Tests.** A single no-op test per repo whose only job is to confirm CTest is wired up correctly.

**Entry conditions.** Empty repos on GitHub (PRIVATE). User has confirmed the GitHub org/user that owns them.

**Exit conditions.** - All three repos exist on GitHub, PRIVATE. - `git clone` of each works. - CI is green on main for all three (no-op test passes). - The three docs (BUILD-ENV.md, NAMING.md, CONTRACT-FORMAT.md) are committed.

**Estimated wall-clock.** 1 day.

**Dependencies.** None.

**Notes for the picking-up session.** - Resist writing math in Phase 0. Bootstrap means bootstrap. - The CMake decision (FetchContent vs submodule for the engine's dep on math core) is load-bearing. Pick once, document in BUILD-ENV.md, do not revisit. Submodules recommended — they pin a SHA. If FetchContent, pin to a tag. - Use a generated .gitignore from gitignore.io. Do not hand-roll. - CI runs on ubuntu-latest and windows-latest. macOS not a target.

---

## Phase 1 — KSTE encoder (pure C reference)

**Goal.** A deterministic, single-threaded reference implementation of the Knowledge-State Tree Encoding pipeline: VHT2 → Möbius → anchor/residual split → tree pack. This is the first piece of real math. It must be readable, slow, and obviously correct. SIMD and parallelization come much later.

**Deliverables.** - shannon-prime-system/include/sp\_kste.h — public API: - `sp_kste_encode(const float *input, size_t n, sp_kste_tree *out)` - `sp_kste_decode(const sp_kste_tree *in, float *output, size_t n)` - `sp_kste_tree_init`, `sp_kste_tree_free` - opaque `sp_kste_tree` struct with the documented 60-node budget and 14-anchor layout - shannon-prime-system/src/sp\_kste.c — implementation. Pure C11. No SIMD intrinsics, no OpenMP, no threads. - shannon-prime-system/src/sp\_vht2.c — the VHT2 transform as its own translation unit so it can be replaced later. - shannon-prime-system/src/sp\_mobius.c — Möbius portion isolated similarly. - shannon-prime-system/tests/test\_kste.c — registers tests T1.1 through T1.5. - shannon-prime-system/docs/KSTE-spec.md — short text description of what each step does, mapping each step to a section of PPT-LAT-Theory.md.

**Tests.** - **T1.1 Determinism.** Encode the same input twice. The two output trees are byte-identical. Proves: no hidden randomness or uninitialised memory dependence. - **T1.2 Frobenius invariance.** Apply the Frobenius automorphism to the input (as defined in Theory section 2.3). The encoded tree's invariants (specifically the per-anchor invariant set documented in the spec) match. Proves: the encoder respects the symmetry the math relies on. - **T1.3 Sign respect.** Negate the input. The encoded tree's sign field changes; the residuals' magnitudes

are unchanged. Proves: residuals are stored in a sign-extracted form. - **T1.4 60-node budget.** No matter what input is fed in (within the spec's input-range constraints), the output tree has at most 60 nodes. Proves: the budget is enforced by construction, not by accident. - **T1.5 14 anchor positions.** Anchors are placed at the 14 positions specified in Theory section 2.5. Proves: anchor placement matches the math, not a re-derivation.

**Entry conditions.** Phase 0 complete. PPT-LAT-Theory.md exists with sections 2.1 through 2.5 stable (these are the sections Phase 1 references).

**Exit conditions.** - 11/11 tests pass on Linux + Windows in CI. (T1.1 through T1.5 yields five logical tests, but they expand into 11 actual test cases — five with random small inputs, three with Frobenius variants, one each for the determinism/sign/budget/anchor invariants.) - The CI run is reproducible: same input file in, same SHA-256 of the output tree, run-to-run.

**Estimated wall-clock.** 5-7 days.

**Dependencies.** Phase 0.

**Notes for the picking-up session.** - The VHT2 transform is described in the legacy theory papers; **do not copy the legacy implementation.** Re-derive from the spec in PPT-LAT-Theory.md and papers/PPT-ARM/. The reason: the legacy code mixes encoder concerns with engine-side caching concerns. We want the math core to know nothing about caches. - The 60-node budget is a hard constraint, not a soft target. If your encoder produces 61 nodes on a pathological input, the test fails and the phase does not pass. Engineer the budget into the algorithm. - Resist adding SIMD or OpenMP. Phase 1 is the reference; later phases optimize. - Float comparisons in tests must use a documented tolerance. Pick 1e-6 for float32 and 1e-12 for float64. Document this in KSTE-spec.md.

---

## Phase 2 — Dominance order and incomparability sieve

**Goal.** Implement the partial order  $\sqsubseteq_d$  on packed KSTE trees and the sieve that maintains a dominance-incomparable set under insertion. The sieve is the data structure the discovery token economy in Phase 11 will be built on, and the gossip layer in Phase 9 needs it to dedup incoming binders.

**Deliverables.** - shannon-prime-system/include/sp\_dominance.h — public API: - sp\_dominance\_compare(const sp\_kste\_tree \*a, const sp\_kste\_tree \*b) returning one of SP\_DOM\_LT, SP\_DOM\_GT, SP\_DOM\_EQ, SP\_DOM\_INCOMP - sp\_dominance\_signature(const sp\_kste\_tree \*t, sp\_dom\_sig \*out) — fast prefilter signature - shannon-prime-system/src/sp\_dominance.c — implementation. Signature-based prefilter, full comparison only on signature collision. - shannon-prime-system/include/sp\_sieve.h — public API: - sp\_sieve\_create, sp\_sieve\_destroy - sp\_sieve\_insert(sp\_sieve \*s, const sp\_kste\_tree \*t) returning SP\_SIEVE\_KEPT, SP\_SIEVE\_DROPPED, SP\_SIEVE\_EVICTED\_OTHERS - sp\_sieve\_iterate(sp\_sieve \*s, sp\_sieve\_visit\_fn fn, void \*ctx) - shannon-prime-system/src/sp\_sieve.cpp — C++ permitted here for std::unordered\_map<sp\_dom\_sig, ...> and intrusive list nodes. Public API stays C. - shannon-prime-system/tests/test\_dominance.c, test\_sieve.c — register T2.1 through T2.10.

**Tests.** - **T2.1 Reflexivity.** compare(t, t) == SP\_DOM\_EQ for any tree. - **T2.2 Antisymmetry.** If compare(a, b) == SP\_DOM\_LT then compare(b, a) == SP\_DOM\_GT. - **T2.3 Transitivity.** If compare(a, b) == SP\_DOM\_LT and compare(b, c) == SP\_DOM\_LT, then compare(a, c) == SP\_DOM\_LT. Tested on a random sample of 10000 triples. - **T2.4 Signature soundness.** If signatures disagree, the full comparison is never SP\_DOM\_EQ. Stronger statement: signature collision is required for equality. - **T2.5 Signature completeness.** Two equal trees always have equal signatures. (T2.4 + T2.5 together let the prefilter be safely used.) - **T2.6 Sieve invariant 1 — incomparability.** After any sequence of insertions, every pair of stored trees

is SP\_DOM\_INCOMP. - **T2.7 Sieve invariant 2 — no duplicates.** No two stored trees are SP\_DOM\_EQ. - **T2.8 Sieve eviction.** Inserting a tree that dominates  $k$  stored trees evicts exactly those  $k$  trees and stores the new one. - **T2.9 Sieve dropping.** Inserting a tree that is dominated by any stored tree returns SP\_SIEVE\_DROPPED and the sieve is unchanged. - **T2.10 Sieve random stress.** 100000 random insertions; invariants T2.6 and T2.7 hold throughout (checked every 1000 inserts).

**Entry conditions.** Phase 1 complete (we need sp\_kste\_tree).

**Exit conditions.** - 10/10 tests pass on Linux + Windows. - T2.10 runs in under 60 seconds on a single core (otherwise the signature prefilter is doing nothing).

**Estimated wall-clock.** 4-5 days.

**Dependencies.** Phase 1.

**Notes for the picking-up session.** - The signature is the most important design decision. Aim for a 16-byte hash that captures the order-relevant structure cheaply. A bad signature does not break correctness (the full compare runs on collision) but a bad signature makes T2.10 slow. - Tree comparisons are the hot path of the sieve. Profile T2.10 once it runs; if signature-collision rate is above ~5% on random input, redesign the signature. - The sieve owns its stored trees (deep copies on insert). Document this in the header. Callers can free their copies after insert returns.

### Phase 3 — ARM (HRR in CRT cyclotomic ring)

**Goal.** Holographic Reduced Representation-style associative memory implemented in a CRT-friendly cyclotomic ring  $\mathbb{Z}_q[x]/(x^N + 1)$ . ARM binds (key, value) pairs by circular convolution and recalls by correlation. This is the substrate the gossip layer (Phase 9) and discovery aggregation will use.

**Deliverables.** - shannon-prime-system/include/sp\_arm.h — public API: - sp\_arm\_create( $N$ , num\_primes, primes[]) - sp\_arm\_bind( $arm$ , key, value) — accumulates a binding into the memory state - sp\_arm\_unbind( $arm$ , key, out\_value) — correlation recall - sp\_arm\_compose( $arm\_dst$ ,  $arm\_src$ ) — ring-additive composition of two ARMs (used by gossip) - sp\_arm\_norm( $arm$ ) — for the renormalization step - shannon-prime-system/src/sp\_arm.c - shannon-prime-system/tests/test\_arm.c — registers T3.1 through T3.6.

**Tests.** - **T3.1 Bind/unbind exact for  $k=1$ .** Bind one key/value pair into an empty ARM. Unbind with the same key. Recall is bit-identical to the original value. - **T3.2 Bind/unbind approximate for  $k=8$ .** Bind 8 random pairs. Unbind each key. Each recall's cosine similarity to the original value is at or above the threshold predicted by the Theory section 4 capacity curve (specifically:  $\cos \geq 0.83$  at  $k=8$  in the documented test setup). - **T3.3 Capacity curve.** Run T3.2 for  $k \in \{1, 2, 4, 8, 16, 32, 64\}$  and record cosine. Stored curve matches Theory section 4 within tolerance. - **T3.4 Negacyclic involution.** The involution  $\text{inv}[N-j] = -\text{in}[j]$  is a ring automorphism; bind/unbind under involution yields the recall under involution. (This is the math invariant the legacy notes flagged as load-bearing.) - **T3.5 Compose distributivity.**  $\text{compose}(arm1, arm2)$  followed by  $\text{unbind}(key)$  equals  $\text{unbind}(arm1, key) + \text{unbind}(arm2, key)$ . - **T3.6 Norm preservation.** Repeated bind operations grow sp\_arm\_norm predictably; the documented renormalization rule keeps norm bounded.

**Entry conditions.** Phase 1 complete (we share the float convention). Phase 2 is **not** a dependency — ARM and the sieve are independent.

**Exit conditions.** - 6/6 tests pass. - The capacity curve from T3.3 is committed as tests/data/arm\_capacity.csv and is read back in CI as a regression check.

**Estimated wall-clock.** 4-5 days.

**Dependencies.** Phase 1 (loose: same code style, same project).

**Notes for the picking-up session.** - The CRT primes for ARM **do not have to be the same primes** as the CRT NTT primitives in Phase 4. Document this clearly in the header. They can be unified later if convenient. - The compose operation is the gossip primitive. Get the math right here; Phase 9 will assume it works. - HRR is sensitive to numerical convention (especially complex-conjugate vs negacyclic). The spec is in PPT-LAT-Theory.md section 4 — implement what is written there, not what you remember from HRR papers.

---

## Phase 4 — CRT NTT primitives

**Goal.** Dual-prime Number-Theoretic Transform with Barrett reduction and CRT recombination. This is the high-throughput multiplication primitive that the engine's attention path (Phase 5) and the sharded-inference demo (Phase 6) depend on. Bit-exactness against a schoolbook reference is the gate.

**Deliverables.** - shannon-prime-system/include/sp\_ntt.h — public API: - sp\_ntt\_create(N, primes[2], roots[2]) - sp\_ntt\_forward(ctx, in, out), sp\_ntt\_inverse(ctx, in, out) - sp\_ntt\_poly\_mul(ctx, a, b, out) — convenience: forward both, pointwise multiply, inverse - sp\_ntt\_crt\_recombine(ctx, residues[2], out) — recombine two prime branches into the integer result - shannon-prime-system/src/sp\_ntt.c — core NTT - shannon-prime-system/src/sp\_barrett.c — Barrett reduction as its own TU - shannon-prime-system/tests/test\_ntt.c — registers T4.1 through T4.5

**Tests.** - **T4.1 Forward/inverse identity.**  $\text{inverse}(\text{forward}(a)) == a$  for random inputs, on each prime branch. - **T4.2 Pointwise multiplication == polynomial multiplication.**  $\text{inverse}(\text{forward}(a) * \text{forward}(b)) == \text{schoolbook\_mul}(a, b) \bmod \text{each prime}$ . - **T4.3 CRT recombination.** A polynomial product computed via CRT-recombine matches the same product computed in big-integer arithmetic, bit-for-bit, across the full output range (no overflow loss). - **T4.4 Barrett correctness.** Random  $(x, p)$  pairs:  $\text{barrett\_reduce}(x, p) == x \bmod p$ . 10 million samples per prime. - **T4.5 Cross-platform bit-exact.** The CSV of test vectors produced on Linux x86\_64 matches the one produced on Windows x86\_64 byte-for-byte.

**Entry conditions.** Phase 1 complete (no direct code dependency, but Phase 4 is in the math core so the conventions need to be established).

**Exit conditions.** - 5/5 tests pass. - Bit-exact cross-platform vectors are committed as tests/data/ntt\_vectors.bin and the CI checks the SHA-256 on both platforms.

**Estimated wall-clock.** 5-6 days.

**Dependencies.** Phase 0 (build environment, since 64-bit-integer behaviour differs between MSVC and gcc and the build env doc pins the compiler).

**Notes for the picking-up session.** - Pick the two primes carefully. They must both be Proth primes of the form  $c \cdot 2^k + 1$  with  $k \geq \log_2(N) + 1$ . Document the selection criteria in sp\_ntt.h. - Avoid \_\_int128 in the public interface. Inside the implementation, prefer two-step Barrett that uses only 64-bit multiplies and one  $64 \times 64 \rightarrow 128$  split that compiles cleanly on both MSVC and gcc. - T4.5 (cross-platform bit-exact) is the kind of test that quietly fails when a compiler optimises an integer operation differently. If T4.5 fails on Windows but passes on Linux, suspect the multiply-high step first. - Do not optimise with SIMD in Phase 4. The reference is the gate; SIMD ports come after the engine is walking.

---

## Phase 5 — Engine bootstrap

**Goal.** Stand up the inference engine. A GGUF loader (using existing libraries — this is engineering, not novel math), a forward pass for a small target model, and the math core wired in: KSTE for KV state, ARM for the latent associative path, NTT for the polynomial-ring attention. Produces logits on a known input that match a documented reference. This phase is the hardest one in the roadmap and the one most likely to slip.

**Deliverables.** - shannon-prime-system-engine/include/spe\_loader.h and spe\_loader.c — GGUF parsing. Use an existing GGUF parser as a vendored dependency; do not write one from scratch. Document which one in BUILD-ENV.md. - shannon-prime-system-engine/include/spe\_model.h and spe\_model.c — model state, tensor lookups, layer iteration. - shannon-prime-system-engine/include/spe\_forward.h and spe\_forward.c — the forward pass: tokenize, embed, transformer layers, output logits. - shannon-prime-system-engine/src/spe\_attention.c — NTT-based attention path using sp\_ntt\_poly\_mul. - shannon-prime-system-engine/src/spe\_kv\_state.c — KSTE-encoded KV state path using sp\_kste\_encode/decode. - shannon-prime-system-engine/src/spe\_arm\_path.c — ARM latent path using sp\_arm\_\*. - shannon-prime-system-engine/cli/sp-engine.c — minimal CLI: sp-engine --model <gguf> --prompt <txt> --tokens N. - shannon-prime-system-engine/tests/test\_forward.c — registers E1. - shannon-prime-system-engine/tests/data/reference\_logits.bin — committed reference logits for the canonical input.

**Tests.** - **E1 Forward pass logits.** Load a small target model (recommended: Qwen 0.5B Q8 or Gemma 0.3B Q8 — pick one and document the choice). Run the forward pass on the canonical input "The quick brown fox". The top-32 logits at the last position match the reference within  $1e-3$  cosine, and the argmax token matches exactly.

**Entry conditions.** Phases 1, 3, 4 complete (KSTE, ARM, NTT). Phase 2 is not strictly required for E1 but should be done by now anyway.

**Exit conditions.** - E1 passes on Linux and Windows. - sp-engine --model <gguf> --prompt "The quick brown fox" --tokens 8 produces deterministic, reproducible output across two consecutive runs on the same machine. - Reference logits are committed and CI checks them every build.

**Estimated wall-clock.** 10-14 days. **This is the highest-variance phase.**

**Dependencies.** Phases 1, 3, 4.

**Notes for the picking-up session.** - The GGUF format is well-documented and there is a reference parser. Use it. Do not write GGUF parsing yourself; it is not where the value of this project lives. - Pick the smallest model you can validate against. Qwen 0.5B Q8 gives plenty of signal and loads in seconds. Resist the temptation to start with a 7B+ model. - The order of operations matters: get a plain (non-SP) forward pass producing correct logits first, **then** swap the attention path to NTT, **then** swap the KV path to KSTE, **then** wire in ARM. Bisect-friendly. - If E1 cosine drifts above  $1e-3$ , do not weaken the threshold. Find the math error. - This is the phase where "I'll just glance at the legacy engine" becomes tempting. Don't. The legacy engine has years of optimization scar tissue that will fight your clean abstractions. Use the legacy engine **only** as a black-box logits oracle to compare against — never as a code reference.

---

## Phase 6 — Two-node CRT-sharded inference demo

**Goal.** Run the engine across two nodes, each computing one prime branch of the CRT decomposition. A driver process recombines. Output is bit-identical to single-machine. Measures



wall-clock vs single-machine to establish a baseline for the sharding overhead.

**Deliverables.** - shannon-prime-system-engine/src/spe\_shard.c — single-prime forward pass mode (driven by a CLI flag `--shard-prime <0|1>`). - shannon-prime-system-engine/cli/sp-engine-shard.c — driver process. Spawns / connects to two shard workers, sends inputs, collects residues, calls `sp_ntt crt_recombine`. - shannon-prime-system-engine/src/spe\_transport.c — TCP transport for residues. Length-prefixed framing. No encryption yet (in-cluster only). - shannon-prime-system-engine/tests/test\_shard.c — registers E2.

**Tests.** - **E2 CRT-sharded bit-identical.** Run the same canonical input through (a) single-machine and (b) two-node shard. The full logits vector (not just top-32) matches bit-for-bit. Wall-clock for both configurations is logged.

**Entry conditions.** Phase 5 complete and E1 has been passing for at least one CI cycle.

**Exit conditions.** - E2 passes. - Wall-clock comparison is recorded in papers/PPT-LAT-Bench-Phase6.md (numbers + setup, not a polished report). - The transport layer is documented as cluster-only: no authentication yet, this is a demo.

**Estimated wall-clock.** 5–7 days.

**Dependencies.** Phase 5.

**Notes for the picking-up session.** - Use loopback (two processes on one machine) for the first runs. Move to two physical machines only when E2 passes on loopback. - The CRT recombine must run after both residues arrive. Build this as explicit synchronization, not “hope the network is fast enough”. Phase 6 establishes the latency reality that informs Phase 12. - Do not try to make Phase 6 fast. Make it correct. Sharding overhead is expected to be significant on a slow network; the value of the demo is bit-identity, not throughput.

---

## Phase 7 — KSTE-encoded crawl cache (single node)

**Goal.** A crawler skeleton that fetches pages, encodes them through KSTE, and inserts them into the dominance sieve. The point is to validate that KSTE compresses real-world text content well, that dominance prunes redundancy as expected, and that the deduplication ratio on a known corpus matches the theoretical prediction.

**Deliverables.** - shannon-prime-lattice/crawler/spcrawl.py (Python is fine here — this is a crawler, not a hot path): - Fetches a URL list with rate limiting, robots.txt compliance, and a user-agent that identifies the project. - For each fetched page, calls into shannon-prime-system via a thin C extension or ctypes to KSTE-encode and sieve-insert. - shannon-prime-system/cli/sp-kste-cli — command-line wrapper for KSTE encode + sieve insert, called from Python. - shannon-prime-lattice/crawler/corpus.txt — list of URLs for the reference corpus (Common Crawl sample, or a curated 10k-URL set). - shannon-prime-lattice/crawler/test\_crawl.py — registers E3.

**Tests.** - **E3 Deduplication ratio.** Crawl the reference corpus. Measure the deduplication ratio (inserts attempted vs inserts that ended in `SP_SIEVE_KEPT`). Compare to the theoretical prediction from Theory section 7. Within the documented tolerance band.

**Entry conditions.** Phases 1 and 2 complete (KSTE + sieve).

**Exit conditions.** - E3 passes. - The reference corpus list and the recorded dedup ratio are committed. - robots.txt is honoured and the user-agent is unique; this is documented.

**Estimated wall-clock.** 4–6 days.

**Dependencies.** Phases 1, 2.

**Notes for the picking-up session.** - The crawler is a means to an end; do not over-engineer it. Rate limit at 1 req/sec per host. Skip non-HTML. - KSTE expects normalized text input. Document the normalization step (HTML strip, Unicode NFKC, lowercase optional). - The dedup ratio is the headline number from this phase. Expect a wide band; the goal is “matches the prediction within tolerance”, not “achieves a specific number”.

---

## Phase 8 — DHT + position-as-arithmetic crawl assignment

**Goal.** A Kademlia-variant DHT where the key space is the position-as-arithmetic lattice from Theory section 6. Nodes are assigned URL crawl responsibilities by their distance to the URL’s lattice key. Multi-node test on loopback first, then on the network.

**Deliverables.** - shannon-prime-lattice/dht/spdht/ — Python package implementing the DHT. (Or Rust if the contributor is comfortable; Python is fine for the lattice prototype.) - node.py — node-side: routing table, RPC handlers, periodic refresh - key.py — URL → lattice key derivation (this is where position-as-arithmetic lives) - assignment.py — given a URL, which N nodes are responsible - shannon-prime-lattice/dht/spdht/test\_dht.py — registers E4 - shannon-prime-lattice/dht/docs/PROTOCOL.md — wire format

**Tests.** - **E4 Load balancing under skewed URL distribution.** Spin up 8 nodes (in 8 processes). Insert 100k URLs drawn from a heavy-tailed distribution (Zipf alpha=1.1, simulating real web traffic). Measure the assignment distribution. Gini coefficient on per-node URL count is below the documented threshold (target: < 0.3).

**Entry conditions.** Phase 7 complete (we need a crawler to drive the DHT).

**Exit conditions.** - E4 passes on loopback (8 processes, one machine). - The wire protocol is documented and stable; future phases will assume it.

**Estimated wall-clock.** 7-10 days.

**Dependencies.** Phases 1 (KSTE is in the assignment key), 7.

**Notes for the picking-up session.** - The “position-as-arithmetic” key derivation is the novel piece. It is described in Theory section 6. Read it carefully before writing key.py. - Use an existing Kademlia library as the starting point for routing tables, but **replace** the XOR metric with the lattice metric. Most Kademlia libraries assume XOR; you will be replacing the core distance function. - Test E4 on loopback. Network tests come in Phase 12.

---

## Phase 9 — ARM gossip aggregation

**Goal.** Nodes periodically exchange compressed ARM state. The capacity-decay rule keeps memory bounded. Aggregated state across the network converges in the sense that a recall against any node, after sufficient gossip rounds, returns the same answer as a recall against the union of all node states.

**Deliverables.** - shannon-prime-lattice/gossip/spgossip/ — Python package: - gossip.py — periodic exchange driver, partner selection from DHT routing table - state.py — local ARM state wrapper (uses sp\_arm\_\* via C extension) - decay.py — capacity-decay rule - shannon-prime-lattice/gossip/test\_gossip.py — registers E5

**Tests.** - **E5 Convergence under gossip.** 16 nodes, each starts with a different set of bindings. Run gossip for K rounds. After K rounds, recall against any node for any of the originally inserted keys returns the expected value within the capacity-curve tolerance. K is documented; expected to be O(log N) but measure.

**Entry conditions.** Phases 3 (ARM), 8 (DHT for partner selection).

**Exit conditions.** - E5 passes. - Memory per node is bounded under continuous bind+gossip — verified by a 24-hour soak test recorded in papers/PPT-LAT-Soak-Phase9.md.

**Estimated wall-clock.** 5–7 days.

**Dependencies.** Phases 3, 8.

**Notes for the picking-up session.** - The capacity-decay rule is what stops memory from growing without bound. Get the rule from Theory section 8; do not invent one. - Partner selection from the routing table is a design decision: pure-random vs distance-weighted. Document the choice and the reasoning. - The 24-hour soak test is non-negotiable. Gossip protocols that look fine in a 5-minute test often pathologically diverge over hours.

---

## Phase 10 — Verification layer

**Goal.** Before the token economy can be built, contributions must be verifiable. This phase implements: (a) commitment scheme for KSTE trees so a node can prove “I had this tree at time T”, (b) dominance proof checks so a verifier can confirm a  $\square_d$  b without re-encoding, (c) slashing simulation under a documented adversarial model.

**Deliverables.** - shannon-prime-system/include/sp\_commit.h and sp\_commit.c — KSTE tree commitment scheme (Merkle-style over the tree nodes). - shannon-prime-system/include/sp\_dproof.h and sp\_dproof.c — dominance proofs: a proof is a sequence of signature checks plus the witness path through the tree. - shannon-prime-lattice/verifier/spverifier/ — verifier daemon (Python is fine). - shannon-prime-lattice/verifier/test\_slash.py — slashing simulation.

**Tests.** - **E6 FP/FN rates under adversarial bindings.** A documented adversarial set (in papers/PPT-LAT-Theory.md section 9): a list of attacks the verifier should catch. Run each attack against the verifier. Measure false-positive rate (legitimate contributions wrongly slashed) and false-negative rate (attacks not caught). Both within documented bands: target FP < 1%, FN < 5%.

**Entry conditions.** Phases 1, 2, 9 complete.

**Exit conditions.** - E6 passes. - Adversarial test set is committed. - A short threat model doc is committed at papers/PPT-LAT-Threat-Model.md.

**Estimated wall-clock.** 7–10 days. **Second-highest-variance phase after Phase 5.**

**Dependencies.** Phases 1, 2, 9.

**Notes for the picking-up session.** - Threat modelling first. Do not write commitment code until the threat model is written down and reviewed. The shape of the commitment depends on what you are defending against. - Dominance proofs that require re-encoding are no proofs; the verifier needs a path through the existing tree using only signatures + a witness. This is the design challenge. - Slashing is simulated only in Phase 10. Real slashing (with tokens) comes in Phase 11.

---

## Phase 11 — Token economy simulator

**Goal.** Two-token economy: a stake token and a discovery token. Discovery tokens are minted when a contribution moves the dominance-incomparable frontier (i.e., the contribution lands in the sieve as SP\_SIEVE\_KEPT AND survives verifier review). Stake is required to participate and

is slashable on verification failure. Simulator runs over a synthetic population of contributors and measures equilibrium properties.

**Deliverables.** - shannon-prime-lattice/economy/speconomy/ — simulator package: - to-kens.py — issuance, balance, slashing rules - contributors.py — synthetic contributor population with documented behaviour distributions - simulate.py — run a multi-epoch simulation - shannon-prime-lattice/economy/test\_economy.py — registers E7

**Tests.** - **E7 Equilibrium under contributor mixes.** Run the simulator for 1000 epochs across (a) all-honest, (b) 10% adversarial, (c) 50% adversarial, (d) lazy-honest majority (mostly low-effort contributions). For each mix, measure: token distribution Gini, fraction of slashable events caught, total discovery-token issuance rate. All within documented bands.

**Entry conditions.** Phases 2, 7, 10 complete.

**Exit conditions.** - E7 passes for all four contributor mixes. - Results are committed at papers/PPT-LAT-Economy-Phase11.md.

**Estimated wall-clock.** 5-7 days.

**Dependencies.** Phases 2, 7, 10.

**Notes for the picking-up session.** - This is a simulator, not a blockchain. No on-chain anything in Phase 11. - The contributor behaviour distributions matter more than the simulation engine itself. Document them carefully in the test. - “Equilibrium” here means “no runaway in token issuance, no degenerate concentration in any one node, no exploit that lets adversaries mint discovery tokens without contributing dominance-incomparable trees”. Measure all three.

---

## Phase 12 — End-to-end three-node pilot

**Goal.** Real machines. Three nodes on three different physical hosts, each running the full stack: crawler, KSTE, sieve, ARM gossip, sharded-inference shards, verifier. Measure throughput, latency, dedup ratio, and token issuance fairness. This is the integration phase; nothing new is being built, but everything has to work together.

**Deliverables.** - shannon-prime-lattice/pilot/ — orchestration scripts (Ansible / shell — pick one and document): - bootstrap.sh — provision a fresh node - start.sh / stop.sh — start/stop the daemons - harvest.sh — pull metrics from a running pilot - shannon-prime-lattice/pilot/dashboard/ — minimal HTML dashboard (or a Markdown report generator) that shows: nodes alive, throughput, latency, dedup, fairness. - papers/PPT-LAT-Pilot-Phase12.md — full pilot report.

**Tests.** No new unit tests. The pilot itself is the test. It either runs for the documented duration with all metrics within the documented bands, or it doesn’t.

**Entry conditions.** Phases 0 through 11 all complete and all green in CI.

**Exit conditions.** - Three real nodes run the full stack for at least 72 hours without manual intervention. - All four headline metrics (throughput, latency, dedup ratio, token issuance fairness) are measured and recorded in the pilot report. - The pilot report includes the start-from-zero runbook so a fourth machine could be added by following written instructions.

**Estimated wall-clock.** 10-14 days.

**Dependencies.** Everything.

**Notes for the picking-up session.** - Phase 12 is integration, not invention. If you find yourself writing new functionality in Phase 12, stop and add it to the appropriate earlier phase first. - The 72-hour soak is non-negotiable. Most systems look fine in a 1-hour pilot and fall over

by hour 18. - The runbook is the deliverable. A pilot that works once but cannot be reproduced is not a pilot.

---

### 3. The contract system

Each phase's **Deliverables** and **Tests** sections together form its contract.

1. A phase is complete only when every file in its deliverables list exists on main of the named repo, and every named test is passing in CI on Linux and Windows.
2. "Passing in CI" means a green workflow on the commit that marks the phase complete. Green local builds are not sufficient.
3. If a deliverable list is incomplete after the fact, reopen the phase and finish it — do not paper over the gap in a later phase.
4. The contract is **append-only**. You may add deliverables; you may not remove them. Unnecessary deliverables get marked "skipped — see session state".
5. Phases complete in order. You may prototype a later phase to clarify an earlier phase, but prototypes do not count toward completion.

The gates are the only mechanism preventing drift between what was supposed to be built and what got built.

---

### 4. The offload pattern

At the end of every working session, write `SESSION-STATE-lat-<phase>.md` to `D:\F\shannon-prime-repos\shannon-prime-lattice\papers\`. Multiple session-state files per phase are expected; future sessions read the most recent first.

#### Required sections:

1. **Date and phase.** ISO date, phase number, working title.
2. **Where I stopped.** Last commit, last test passing, next file to edit.
3. **What's green.** Tests passing right now.
4. **What's red.** Tests failing, with error and best guess at cause.
5. **What I learned.** Design decisions not yet documented elsewhere, blind alleys to skip, departures from spec that need either fixing or spec update.
6. **What the next session should do first.** One paragraph, concrete action.

Sessions are short, the project is long. When picking up, **read the most recent session-state file before the roadmap**. The roadmap says what to build; the session state says where things are.

---

### 5. Anti-contamination rule

This is the rule that this project lives or dies by.

**Sessions MUST NOT copy code or designs from `D:\F\shannon-prime-repos\shannon-prime\` or `D:\F\shannon-prime-repos\shannon-prime-engine\`.** Those repos are reference for **theory only** (the math papers in `papers/PPT-ARM/`) and are explicitly off-limits for everything else.

Specifically:

- Do not open .c or .h files in the legacy repos.
- Do not open CMakeLists.txt or build scripts in the legacy repos.
- Do not skim test files in the legacy repos. (Yes, even tests. Tests encode design assumptions that may be wrong for the new architecture.)
- Do not “just check how they did it”. The whole point of this rebuild is to be free of the legacy code’s accumulated decisions.

What you **may** read:

- The math papers in papers/PPT-ARM/ (these are the theory reference).
- This roadmap.
- PPT-LAT-Theory.md.
- PPT-LAT-Systems.md.
- The legacy engine as a **black box**: you can run it, observe its outputs, and use those outputs as comparison oracles. You cannot look inside.

Why the rule is strict: the user has reported that previous attempts to start fresh got contaminated by the existing codebase being pulled back in piece by piece. The result was that the “new” repo became indistinguishable from the old one within a few sessions. The memory entry `feedback_no_cross_contamination` is the canonical statement of this; it is binding.

If you are in doubt about whether a particular read is allowed, the answer is: **don’t**. Re-derive from the math papers.

---

## 6. Testing discipline

Tests are the contract. Tests that pass in phase N must continue to pass in phase N+k for all k. Concretely:

- Every test from every prior phase runs in CI on every commit. There is no “this test was for the old API, we’ll skip it now”. If an API changes, the test is updated to track the new API while still proving the same invariant.
- A regression — a previously-green test going red — blocks the commit. CI must be green on main at all times.
- If a regression is found in a later phase, the root-cause fix goes in the earliest phase whose module is responsible, and a regression test is added that would have caught it earlier.
- Test data files (reference logits, NTT vectors, capacity curves, dedup ratios) are committed to the repo. They are part of the contract.

The cost of this discipline is that early-phase tests must be designed to be stable. Do not write tests against implementation details; write tests against the invariants in the theory paper. If the theory paper changes, that is a coordinated change; if the implementation changes, the tests must not.

---

## 7. GitHub workflow

All three repos are **PRIVATE** on GitHub. They will stay private through Phase 12.

**Per-phase workflow:**

1. Pick up phase N. Read the latest `SESSION-STATE-lat-N.md` if any.
2. Work on a feature branch named `phaseN/<short-description>`.
3. Commit small, named commits. Each commit message starts with `[phaseN]`.

4. When the phase's exit conditions are all met, open a pull request from the feature branch to main. The PR description includes the contract check: every deliverable checked off, every test listed with its status.
5. CI must be green on the PR. Merge to main.
6. Tag main with phaseN-complete after merge.
7. Push the tag.
8. Write the final SESSION-STATE-lat-N.md describing the completion.

If a phase takes multiple sessions, only the **final** session merges to main. Intermediate sessions push to the feature branch and write intermediate session-state files. This is the offload pattern integrated with the GitHub workflow.

Each repo's CI runs only its own tests. The lattice repo's CI runs the orchestration/Python tests; the math core's CI runs the C unit tests; the engine's CI runs the engine tests and the E1-E2 integration tests. There is no global CI; the gate is that each repo is green at the moment of phase completion.

---

## 8. Risks and Mitigations

A short, blunt list. Read this before starting each phase.

**Risk 1 — Phase 5 schedule slip.** The engine bootstrap phase is doing four things at once (GGUF loader, plain forward pass, math-core integration, reference validation). Any one of them can stall the whole phase. *Mitigation.* Sequence them. Plain forward pass first with the **existing** attention math (not NTT yet). Validate against the reference logits. Then swap in NTT attention. Validate again. Then KSTE KV. Then ARM. Each swap is a bisect point. If a swap breaks the cosine target, you know which swap.

**Risk 2 — Anti-contamination drift.** The legacy repos are right there. The temptation to “just peek” compounds over many sessions. *Mitigation.* Each session's first action when picking up a phase is to re-read Section 5 of this doc. Yes, every session. If a session does open a forbidden file, it must document the contamination in its session-state file and flag the deliverables that may have been influenced.

**Risk 3 — Verification (Phase 10) is harder than it looks.** Designing a commitment scheme and a dominance-proof system without re-encoding is a real piece of cryptographic engineering, and the project does not have a cryptographer on it. *Mitigation.* Budget extra time. Start the threat model writeup before any code. If the FP/FN bands in E6 cannot be hit with the proposed scheme, escalate — do not weaken the bands. Consider engaging outside review before committing.

**Risk 4 — Gossip convergence in Phase 9.** Gossip protocols look easy in a small test and pathologically diverge at scale or over time. *Mitigation.* The 24-hour soak is mandatory. Do not skip it under schedule pressure. If you are tempted to skip it, you are exactly the person who needs to run it.

**Risk 5 — Phase 12 pilot logistics.** Three real machines on a real network introduce failure modes (clock skew, partial network partitions, ISP-level filtering of the user agent) that loopback testing cannot reproduce. *Mitigation.* Start Phase 12 with a “day zero” test on one machine only, then add the second, then the third. Don't go to three machines on the first day. The runbook is built up incrementally with each machine added.

**Risk 6 — Cross-platform bit-exactness drifting silently.** The math core is supposed to be bit-exact on Linux and Windows. A compiler change or a Windows update can break this without any local signal. *Mitigation.* T4.5 (cross-platform bit-exact vectors) runs on every CI

build, on both platforms. If T4.5 ever goes red, treat it as a blocker for everything downstream; do not paper over by regenerating the vectors on the failing platform.

**Risk 7 — Token economy gaming in Phase 11.** A simulated economy can hide attacks that a real economy would surface. The simulator may declare equilibrium under a contributor mix that is, in reality, a degenerate case. *Mitigation.* The four contributor mixes in E7 are a floor, not a ceiling. Add adversarial mixes that target the specific incentive structures of the two-token system. Document each new mix in the test file with the attack it represents.

**Risk 8 — Theory drift.** The theory papers are the spec. If theory changes mid-stream, every downstream phase needs reconciling. *Mitigation.* Theory changes land with a list of impacted phases and tests, updated in the same commit. Check theory papers' git log first when picking up a phase.

---

## 9. Where to start tomorrow

First session after this roadmap lands:

1. Re-read Section 5 (anti-contamination) and Section 4 (offload pattern).
2. Open Phase 0. Create the three GitHub repos (PRIVATE).
3. Push the bootstrap files: .gitignore, LICENSE, README.md, BUILD-ENV.md, NAMING.md, CONTRACT-FORMAT.md, empty CMakeLists, placeholder CI workflow.
4. Verify CI is green on all three repos.
5. Write SESSION-STATE-lat-0.md, commit, push.

Phase 1 starts on the next session, with a clean slate and a clear contract.